

Kalpi Builder Assignment

Role: Backend/Infra Engineer (Kalpi Builder) **Company:** Kalpi – India's First Systematic Quant Investing Platform **Time Allowance:** 24 Hours from receipt

Context

At Kalpi, our core engine relies on executing systematic, quantitative, factor-based portfolios flawlessly. As a Kalpi Builder, you will be responsible for the infrastructure that connects our quantitative models to the real world.

Problem Statement One:

Design and implement an end-to-end **Portfolio Trade Execution Engine**.

The engine must take a desired portfolio state (a list of stocks and their target quantities), authenticate with a user's stock broker, and automatically execute the necessary trades in a single click.

Core Workflows & Requirements

1. Broker Integration & Authentication

- The system must support at least 5 major Indian brokers (e.g., Zerodha, Fyers, AngelOne, Groww, Upstox/Indiabulls).
- Implement a standardized "Adapter Pattern" or interface so adding a 6th broker requires minimal code changes.
- **Note on Open Source:** If you prefer to use an existing, reliable open-source module to normalize broker connections rather than building your own from scratch, you are welcome to do so. However, you must provide proper justification for your choice and be able to explain its underlying mechanics thoroughly.

2. Execution Logic (The Core Engine) The engine must determine the "delta" between the user's current holdings and the new target portfolio.

- **First-Time Portfolio:** If the user has no existing holdings, simply execute "BUY" orders for the listed stocks and quantities.
- **Portfolio Rebalancing:** In the case of rebalancing an existing portfolio, the engine *does not* need to calculate the delta. The input payload will directly provide explicit instructions, such as:
 - **SELL:** Specific quantities of existing stocks to exit.
 - **BUY (New):** Specific quantities of new stocks to purchase.

- **REBALANCE (Adjust):** Specific quantity changes (buy/sell) for existing overlapping stocks.

3. Notification System

- Once the execution (first-time or rebalance) is complete, trigger a notification back to the consumer (this can be a mocked Webhook, WebSocket event, or a simple email/console log for the sake of the assignment) summarizing the executed trades and any failed orders.

Technical Specifications

- **Backend Framework:** Python (FastAPI).
- **Deployment:** Must be fully containerized using Docker (provide a `Dockerfile` and `docker-compose.yml`).
- **Architecture:** Use the best architectural practices for trading systems (e.g., modularity, clear separation of concerns, robust error handling for API rate limits and failed trades).
- **Bonus (Brownie Points):** A basic frontend interface (in any framework/language) to test the flow visually (upload target portfolio -> connect broker -> click execute -> view results).

Submission Guidelines

- Push your code to a public GitHub repository and share the link.
- Include a comprehensive `README.md` containing:
 - Setup and run instructions (Docker commands).
 - A brief explanation of your architectural choices and how the rebalance logic works.
 - Justification for any third-party open-source trading libraries used.
- **Deadline:** 1 day (24 hours) from the time you receive this prompt.

Problem statement 2: Multi-Frequency Financial Data Platform

This exercise asks you to design the architecture for a platform that handles the **NSE + BSE Universe (5,000+ symbols)** across four distinct temporal frequencies.

1. The Context: The "Temporal Join" Problem

A single metric like the **P/E Ratio** seems simple: $\text{Price} / \text{Earnings Per Share (EPS)}$. However, in a production quant environment, the "ingredients" move at different scales:

Frequency	Data Type	Examples	Arrival Rate
Live (Tick)	Broker WebSocket	LTP, Volume	~300ms per symbol (5k symbols)
Daily (EOD)	Market Close	Adjusted Price, 52w High	Once per day (6:00 PM)
Quarterly	Fundamentals	EPS (TTM), Revenue	Irregular (Earnings Season)
Annual	Balance Sheet	Book Value, Dividend	Once per year

The Conflict: * **Live View:** Users need to see the P/E ratio updating in real-time as the price ticks, using the most recent EPS.

- **Historical View:** To plot a 5-year P/E chart, you must join historical prices with the *exact* EPS value that was "known" to the market on that specific date. Using today's EPS for a 2022 price point is a "look-ahead bias" that ruins quant models.

2. Your Task: Technical Design Document

Please prepare a design document (approx. 1000–1500 words) answering the following architectural questions. We value **specific technology choices** and **justification** over generic buzzwords.

Q1: Storage (Four Frequencies, One System)

- Describe your storage stack. What databases do you use for Tick data vs. Fundamentals?
- **Point-in-Time Correctness:** If a company releases Q3 results at 7 PM on a Tuesday, how do you version that data so historical calculations stay accurate to what was known *at that time*?
- **Archival:** 5,000 symbols generating ticks for years is petabyte-scale data. What is your retention and downsampling (OHLC) strategy?

Q2: The Compute Engine (Derived Metrics)

- **The Trigger:** When a new tick arrives, P/E changes. When a new EPS arrives, P/E changes. Do you compute on-write, on-read, or on-schedule?

- **Note on Latency:** For live updates, we do not require sub-millisecond tick-by-tick propagation. Second-level latency is acceptable for the user-facing UI.

Q3: APIs & Serving (Low Latency)

- **Pattern A (Live):** A user opens a stock page and needs current metrics (P/E, Momentum, Yield) in <200ms.
- **Pattern B (Historical):** A user requests a 3-year P/E chart (~750 data points).
- What is your API strategy and caching/invalidation logic for both patterns?

Q4: The Full Picture

Provide a high-level architecture diagram tracing a single **P/E Ratio** from:

1. The raw WebSocket tick to the live UI (optimized for second-level latency).
2. The raw historical EOD price to a 5-year chart.

3. Evaluation Criteria

We are looking for a "Founding Engineer" mindset:

1. **Systems Thinking:** Can you handle the scale of 5,000 symbols without the backend melting?
2. **Financial Nuance:** Do you understand why "Point-in-Time Correctness" is the difference between a real quant platform and a hobby project?
3. **Pragmatism:** Are you choosing tools because they are "cool" or because they solve the specific latency/throughput trade-offs of this problem?

Questions or Clarifications? Ping us at info@kalpicapital.com or message at +918871911901.